Repository Generation via Coding-Testing Loop

Zhaoxi Zhang
Peking University
zhaoxizhang25@stu.pku.edu.cn

Eddie Lin
Peking University
2501213379@stu.pku.edu.cn

Junhan Gong
Peking University
2501111442@stu.pku.edu.cn

Abstract

Large Language Models (LLMs) have recently shown remarkable progress in code generation, yet their ability to construct complete software repositories from scratch remains poorly understood. In this work, we use RepoZero, the first benchmark that enables fully automated, execution-based verification of repository-level generation from scratch, to propose an Agentic Code-Test Evolution (ACE) framework that performs iterative test generation and error-driven refinement, enabling effective test-time scaling for repository-level synthesis. Extensive experiments across multiple state-of-the-art LLMs and agent frameworks reveal that even the strongest LLM agents achieve only limited pass rates (30% - 55%), exposing a substantial gap between current capabilities and real-world software development requirements. Code is available at https://github.com/JesseZZZZZ/RepoZero.

1 Introduction

Recent advances in LLM-based coding have drawn increasing attention toward fully autonomous end-to-end software development. State-of-the-art large language models (Liu et al., 2025a; Yang et al., 2025; Team et al., 2026; Xiao et al., 2026; Zeng et al., 2026) have been integrated into coding agents (Wang et al., 2024; Gao et al., 2025; Zhang et al., 2025) capable of completing complex programming tasks with minimal or no human intervention. From the perspective of benchmark difficulty, coding tasks can generally be categorized along two orthogonal dimensions. The first dimension concerns the scope of code modification, including: (1) single-file coding and (2) repository-level coding. The second dimension concerns the nature of the programming task itself, including: (1) debugging, (2) incremental development, and (3) generation from scratch.

Current coding agents have largely mastered single-file editing. However, evaluating repository-level modifications remains a burgeoning frontier. Benchmarks such as SWE-bench (Jimenez et al., 2024), Multi-SWE-bench (Zan et al., 2025a), and FEA-bench (Li et al., 2025) have pioneered the assessment of repo-level edits and incremental development in real-world scenarios. While these benchmarks leverage patch-based verification and unit testing, the domain of from-scratch repository generation remains relatively unexplored. A primary obstacle is the scarcity of pre-existing unit tests available online for newly generated projects. Consequently, current benchmarks for from-scratch generation, such as CodeS (Zan et al., 2025b), EvoCodeBench (Zhang et al., 2026), and NL2RepoBench (Ding et al., 2026), predominantly rely on human-in-the-loop or LLM-as-a-judge frameworks. These methods, however, introduce subjective biases from both human evaluators and language models, leading to potentially unreliable evaluation outcomes.

Data leakage presents another formidable challenge for repository-level coding benchmarks. Since modern large language models (LLMs) are extensively pre-trained on massive GitHub corpora (Zeng et al., 2026; Xiao et al., 2026), it is increasingly difficult to discern whether their performance stems from genuine reasoning capabilities or mere rote memorization. While some studies (Zhang et al., 2026) attempt to mitigate this by utilizing rare or recently published repositories, this approach inherently constrains the dataset scale, thereby precluding large-scale evaluation. Furthermore, such "clean" datasets are subject to rapid contamination post-publication, necessitating frequent and labor-intensive updates to maintain their integrity.

To address these challenges, we use **RepoZero**, the first verifiable benchmark specifically designed to evaluate the from-scratch repository-level generation capabilities of LLM agents. We introduce an Agentic Code-Test Evolution (ACE) workflow for test-time scaling. Unlike existing frameworks,

where the lack of a verifiable environment makes retrieving ground-truth outputs for LLM-generated tests impossible, RepoZero leverages the source repository as an oracle, providing a deterministic and automated gold standard for evaluation.

In summary, our primary contributions are as follows:

1. **Test-Time Scaling Framework**: We propose a novel agentic framework centered on test-time scaling, which significantly enhances the success rate of complex, repository-level synthesis.

2. **Comprehensive Benchmarking**: We conduct an extensive evaluation of state-of-the-art models and agentic scaffolds, providing critical insights and a robust foundation for future research in autonomous software engineering.

2 Benchmark: RepoZero

2.1 Statistics of RepoZero

RepoZero consists of two subsets: **RepoZero-Py2JS**, where the source repositories are implemented in Python, and the agent is required to reimplement them in JavaScript, and **RepoZero-C2Rust**, where the source repositories are written in C/C++ and the agent is tasked with reimplementing them in Rust. As illustrated in Fig. 1, RepoZero covers a diverse range of repository categories. All repositories are manually curated and further preprocessed via an automated pipeline covering test case generation, environment configuration, and data filtering. We employ LLMs to estimate the difficulty of each sample under several rubrics: Lines of Code (LOC), API counts, and input/output complexity. We apply majority voting (Claude-4.6-Opus, GLM-5, and DeepSeek-V3.2) to obtain the final estimation. The datasets are accordingly partitioned into *Easy*, *Medium*, and *Hard* subsets. From the curated repositories, we construct 400 samples for RepoZero-Py2JS and 200 samples for RepoZero-C2Rust. The source repositories of these samples are manually selected. Because the remaining pipeline is automated after repository selection, the benchmark can be continuously updated to mitigate potential data leakage (once released, RepoZero will inevitably be collected as part of the training data of foundational models).

2.2 Task Definition and Challenges

In a repository reproduction task, an LLM agent is provided with: (1) the functional specifications

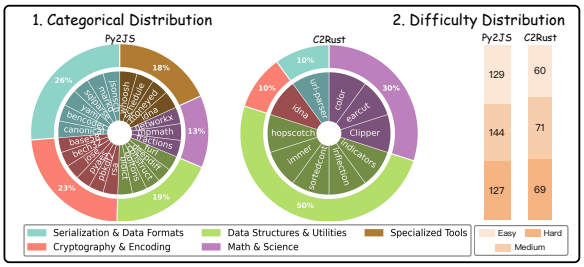

Figure 1: Demonstration of (1) categorical distribution, and (2) difficulty distribution of RepoZero-Py2JS (left) and RepoZero-C2Rust (right).

of the source repository's APIs, (2) four white-box test cases (comprising inputs, ground-truth outputs, and descriptions), and (3) constraints to preclude "shortcut" behaviors, such as cross-language embedding or direct API delegation. The target repository is compiled, where applicable, and evaluated via black-box testing to ensure its outputs strictly align with the source repository's baseline. This task presents four primary challenges for LLM agents: (1) Functional Synthesis, translating high-level API specifications into concrete implementations; (2) Modular Reasoning, resolving inter-module dependencies across multiple files; (3) Iterative Self-Correction, refining code through autonomous test design and execution; and (4) Long-context Retention, maintaining architectural coherence over extensive reasoning horizons. As shown in Sec. 5, current agents exhibit several limitations in navigating these complexities.

2.3 Evaluation Pipeline

We employ two popular coding agents: OpenHands-bash (Wang et al., 2024; Sutawika et al., 2026) and Mini-SWE-Agent (Yang et al., 2024a). These agents can interact with the computer system via a series of tools, for instance, a terminal, where the agent can run commands. The agent is prompted with the task information and prompted to generate a repository with an assigned entry point. After finishing the generation, black-box tests are applied to test the quality of the target repository.

To ensure a rigorous evaluation, we address the potential for "cheating" behaviors—such as accessing ground-truth repositories or manipulating test suites—frequently observed in recent LLM agent benchmarks (Jimenez et al., 2024; Merrill et al., 2026). We implement a multi-layered protocol to maintain the integrity of the evaluation process:

Black-box Testing To mitigate the risk of agents overfitting to specific test outcomes, we implement a black-box evaluation framework. Only four representative white-box test cases are included in the prompt for contextual guidance, while the remainder of the evaluation suite is withheld. By prioritizing semantic correctness over superficial execution results, this methodology ensures that agents cannot bypass rigorous verification through trivial output matching or heuristic shortcuts.

Environment Isolation and Security Evaluation is performed within isolated Docker containers characterized by restricted file-system permissions. To maintain the integrity of the benchmark, the source repository's test files are configured as read-only, and the hidden evaluation suite is never exposed to the agent's accessible file system. These safeguards prevent agents from manipulating or circumventing challenging test cases to artificially inflate performance metrics.

Execution Constraints and Cross-Language Integrity We impose stringent limitations on system-level commands to preclude non-generalizable solutions. Specifically, agents are prohibited from importing external packages into the target repository. Furthermore, we enforce strict language consistency: for instance, if an agent is tasked with porting a Python library (e.g., mpmath) to JavaScript, the use of bridge commands or cross-language invocations (e.g., executing Python scripts via shell wrappers) is strictly forbidden. Such constraints necessitate that agents rely on complex logical reasoning and cross-language synthesis rather than delegating tasks to external APIs.

3 Agentic Code-Test Evolution Workflow

Recent studies have widely acknowledged that jointly generating test cases alongside code can substantially improve code generation performance (Ding et al., 2025). However, the code-test feedback loop has not been broadly adopted in existing workflows for two primary reasons. First, LLMs do not consistently exhibit the tendency to generate test cases without explicit prompting or a predefined procedural framework. Second, test cases generated by LLMs are typically not independently verifiable; consequently, such cases can only assess code executability rather than functional correctness.

Inspired by Agentless (Xia et al., 2024), which replaces fully autonomous agentic procedures with a structured workflow to enhance LLM-based bug localization, we propose the **Agentic Code-Test Evolution** (ACE) workflow, a dedicated framework designed for RepoZero. As illustrated in Fig. 2 (B), ACE adopts an iterative code–test feedback loop.

RepoZero provides a suitable testbed for evaluating ACE. This is primarily because the ground-truth outputs for RepoZero samples are obtained by executing the source repository itself, which enables the generation of an effectively unlimited number of test cases while ensuring the correctness and reliability of their corresponding labels.

In practice, ACE serves as a general framework that can be integrated with arbitrary coding agents. The workflow begins with a coding agent that generates a target repository from scratch based on the APIs of the source repository. Subsequently, a testing agent produces multiple test cases and executes the source repository to obtain the corresponding ground-truth outputs. Notably, a filtering strategy that discards non-deterministic or exception-triggering test cases is also applied at this stage. If the generated repository successfully passes all test cases, the iterative loop terminates. Otherwise, the resulting error messages are fed back to the coding agent, which then revises the generated repository accordingly.

4 Experimental Setup

Models and Scaffolds We evaluate our benchmark using two widely recognized agent scaffolds for software engineering: OpenHands-bash (Sutawika et al., 2026) and Mini-SWE-Agent (Yang et al., 2024a). These frameworks are integrated with state-of-the-art LLMs, including Kimi-K2.5, Kimi-K2.6 (MoonShot, 2026), GLM-5, GLM-5.1 (Zeng et al., 2026), DeepSeek-V3.1, DeepSeek-V3.2 (DeepSeek-AI, 2025), DeepSeek-V4 (DeepSeek-AI, 2026), Ernie-5.0 (Wang et al., 2026), Minimax-M2.5, Minimax-M2.7 (MiniMaxAI, 2026) and Claude-4.6 (Anthropic, 2026). To ensure maximum flexibility, we do not impose an explicit cap on the number of tool-calling iterations; the only constraint is the maximum output token limit inherent to the base models. Furthermore, agents are permitted full access to context management techniques and retrieval-augmented tools to facilitate complex problem-solving.

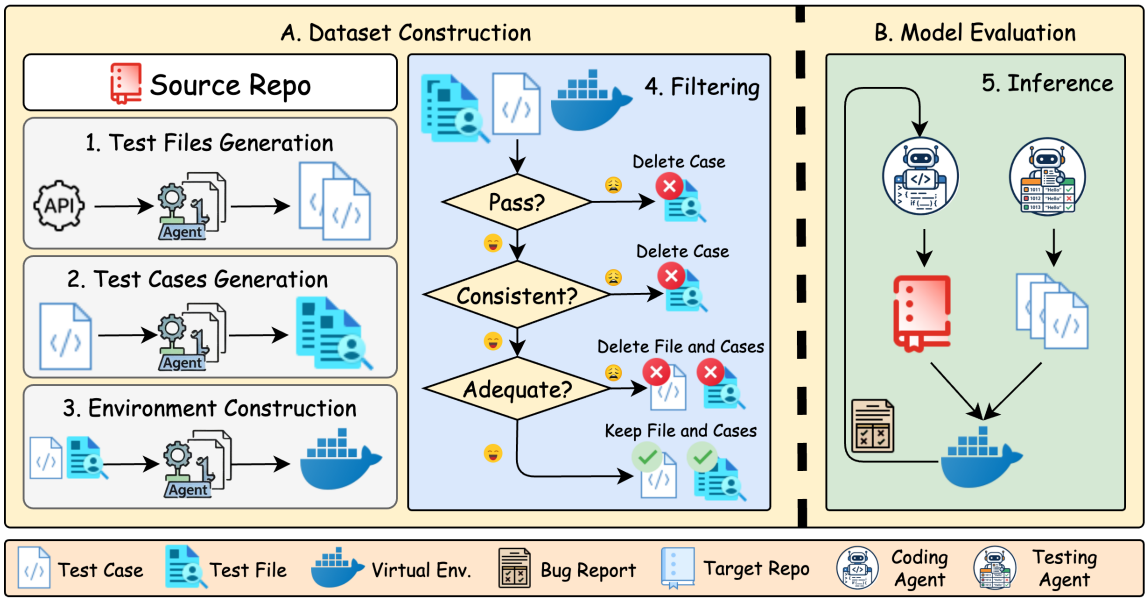

Figure 2: Illustration of (A) the construction of the RepoZero benchmark, and (B) agentic behavior during the evaluation stage. The loop ends when all cases produced by the testing agent succeed.

Metrics We adopt the Pass Rate (PR) as our primary evaluation metric. A sample is considered successful ($PR = 1$) if and only if the outputs from both the source and target repositories exhibit strict string-level consistency; otherwise, it is assigned a value of 0. Additionally, we report the Success Rate (SR) across test cases. While the SR (see Appendix **??**) offers a complementary perspective, the PR is utilized as the more robust and definitive measure of functional equivalence. Results are reported as mean $\pm$ bootstrap standard deviation.

5 Results and Analysis

5.1 Main Evaluation

The primary experimental results for the RepoZero-Py2JS and RepoZero-C2Rust benchmarks are presented in Table 1 and Table 2, while results in Table **??** illustrate the performance across multiple categories. Overall, all evaluated models exhibit suboptimal performance on these benchmarks regardless of whether the OpenHands-bash or Mini-SWE-Agent scaffold is employed. Notably, Claude-4.6-Sonnet consistently achieves the highest performance across all evaluation metrics.

Our analysis reveals that Mini-SWE-Agent generally outperforms OpenHands-bash. This performance gap can likely be attributed to more sophisticated context engineering in Mini-SWE-Agent, whereas OpenHands-bash represents a baseline agent configuration restricted to a rudimentary bash interface. Furthermore, as illustrated in Fig. 4 (right), OpenHands-bash fails even on test cases explicitly provided within the initial prompt. This observation suggests that without robust, carefully designed context management, agents struggle to retain and leverage critical information during complex, long-context reasoning tasks.

Benchmark Validity While the semi-synthetic nature of RepoZero precludes inherent quality guarantees, its empirical validity is substantiated by consistent performance alignment within established model families. Adhering to the $Benchmark^2$ principle (Qian et al., 2026), our results (Tables 1 and 2) demonstrate that successive iterations of the Kimi, GLM, DeepSeek, and Minimax series exhibit progressive performance gains. Furthermore, the performance hierarchy—especially Claude and Ernie—aligns with broader industry benchmarks. This structural consistency across diverse model tiers underscores the reliability of RepoZero.

Cost-Performance Trade-offs Figure 3 illustrates the general correlation between inference expenditure and model efficacy, where the pass rate typically scales with per-sample cost. While Claude-4.6-Sonnet defines the upper bound for both metrics, this relationship is not strictly monotonic. Notably, GLM-5.1 significantly outperforms GLM-5 despite a nearly identical cost structure, highlighting how architectural advancements can decouple performance from computational overhead. However, the persistent performance gap be-

Table 1: Evaluation with Openhands-bash on RepoZero. We present the pass rate of models across Easy, Medium, and Hard task difficulties.

Model	Py2JS				C2Rust			
	Easy	Medium	Hard	**Avg.**	Easy	Medium	Hard	**Avg.**
GLM-5	37.21	23.61	22.05	27.46±2.29	31.67	20.00	20.29	23.47±2.34
GLM-5.1	46.51	38.89	16.54	34.19±2.38	45.90	31.43	28.99	35.05±2.19
Kimi-K2.5	36.36	33.02	24.74	31.30±2.97	40.98	30.00	28.99	33.00±1.97
Kimi-K2.6	47.29	27.78	25.20	33.25±2.36	50.82	30.00	36.23	38.51±2.03
Ernie-5.0	27.91	18.75	11.81	19.46±2.02	22.95	12.86	4.35	13.97±2.36
Minimax-M2.5	29.97	22.23	18.89	22.72±1.92	36.07	18.57	31.88	28.48±2.31
Minimax-M2.7	34.88	29.17	16.54	27.15±2.15	37.70	25.71	33.33	32.12±2.91
DeepSeek V3.1	34.11	25.69	18.11	26.08±2.26	43.33	30.00	26.09	32.49±2.00
DeepSeek V3.2	37.21	31.25	18.11	29.01±2.34	48.33	28.57	26.09	33.55±2.52
DeepSeek V4 Flash	38.76	32.64	20.47	30.74±2.33	40.98	34.92	31.88	35.41±2.38
DeepSeek V4 Pro	48.84	38.19	32.28	39.78±2.51	40.98	32.86	33.33	35.53±2.40
Claude-4.6-Sonnet	59.69	49.31	45.67	51.46±2.53	53.33	45.71	36.23	44.60±2.74

Table 2: Evaluation with Mini-SWE-Agent on RepoZero. We present the pass rate of models across Easy, Medium, and Hard task difficulties.

Model	Py2JS				C2Rust			
	Easy	Medium	Hard	**Avg.**	Easy	Medium	Hard	**Avg.**
GLM-5	55.81	43.75	37.01	45.52±2.48	49.18	37.14	30.43	38.51±1.98
Kimi-K2.5	44.19	32.64	24.41	33.76±2.48	54.10	40.00	37.68	43.53±2.33
Ernie-5.0	34.11	15.28	13.39	20.77±2.08	26.23	14.29	17.39	19.97±2.04
Minimax-M2.5	49.91	29.25	22.68	30.63±2.72	32.79	24.29	20.29	25.39±2.31
DeepSeek V3.1	58.91	45.14	26.77	43.76±2.50	50.82	37.14	31.88	39.45±2.51
DeepSeek V3.2	52.71	44.44	35.43	44.20±2.59	49.18	38.57	26.09	37.41±2.78
Claude-4.6-Sonnet	65.12	54.17	44.88	54.70±2.55	59.02	45.71	39.73	47.58±2.90

tween these leading open-source models and SOTA closed-source systems (e.g., Claude) remains a primary challenge for open-source development.

5.2 Test-time Scaling via ACE

Table 3 summarizes the ACE (a case study is provided in Fig. 4 left) performance of OpenHands-bash and Mini-SWE-Agent on the RepoZero-Py2JS benchmark, both of which are underpinned by the DeepSeek V3.1 backbone. The empirical results demonstrate a substantial performance gain when test cases are dynamically generated and executed during the inference phase. These findings from the ACE framework underscore two pivotal directions for the evolution of repository-level coding agents: (1) generating high-quality, executable, and verifiable test cases; and (2) integrating predefined test cases—whether human-authored or LLM-generated—in automated software engineering workflows.

5.3 Failure Analysis

We categorize the identified failure modes into four distinct groups: (1) failure to generate the executable code; (2) failure to pass white-box test cases (which are provided to the agent as illustrative examples); (3) general runtime errors, such as arithmetic overflow or dependency conflicts; and (4) output mismatches between the source and target repositories during black-box testing.

The performance breakdown of open-source models with the OpenHands-bash scaffold is illustrated in Fig. 4 (right). Our analysis reveals that runtime errors occur infrequently, whereas other failure modes predominate. Based on these observations, we offer the following insights for the design of future autonomous coding agents:

Long-term Memory and Context Retention A non-negligible portion of failures was observed even within white-box test cases, a phenomenon primarily attributable to deficiencies in the agents' long-context management. Despite the inclusion of all white-box constraints in the initial prompt,

Table 3: Evaluation on **Py2JS** with DeepSeek V3.1 as the backbone model. We present the pass rate of models across Easy, Medium, and Hard task difficulties, and the maximum retry times are set to 0 (coding only), 1 (coding-testing-refining), and 2 (coding-testing-refining-testing-refining).

Method	Openhands-bash				Mini-SWE-Agent			
	Easy	Medium	Hard	**Avg.**	Easy	Medium	Hard	**Avg.**
Retry-0	34.11	25.69	18.11	26.08±2.26	58.91	45.14	26.77	43.76±2.50
Retry-1	45.74	37.50	25.20	36.15±2.38	58.91	47.92	29.92	45.67±2.44
Retry-2	51.14	41.51	37.11	42.88±2.93	64.34	51.39	40.16	52.06±2.38

agents frequently exhibit "contextual drift" during extended reasoning sequences, losing track of critical requirements as the trajectory length increases.

Autonomous Environment Utilization We observe that most high-performing agents proactively design and execute automated unit tests within the provided environment. This self-correcting capability is instrumental in mitigating runtime errors, which are infrequent across our evaluation suite. Integrated with the empirical data in Table 3, these findings underscore that the ability to leverage an executable environment for test-driven development is a fundamental determinant of success in complex software engineering tasks.

Discrepancy Between Runnability and Correctness While many existing benchmarks evaluate repository-level performance based solely on execution success, our results expose a substantial gap between runnability and semantic correctness. Notably, approximately 40% of the executable code generated by agents fails to match the deterministic output of the source repository. This discrepancy reinforces the necessity of the more stringent, output-verified evaluation framework of RepoZero.

6 Related Works

6.1 Repo-level Coding Benchmarks

Traditional coding benchmarks (Chen et al., 2021; Lu et al., 2021; Gu et al., 2024) primarily assess the capability of LLMs to generate isolated code snippets, a scope that fails to capture the complexities of real-world software engineering. To bridge this gap, SWE-bench (Jimenez et al., 2024) introduced the first evaluation framework for repository-level bug fixing. This line of research has been further extended by Multi-SWE-bench (Zan et al., 2025a), SWE-bench-Multimodal (Yang et al., 2024b), and SWE-bench-Pro (Deng et al., 2025), which incorporate multiple modalities and diverse programming languages. Complementing the focus on maintenance, FEA-bench (Li et al., 2025) emerged as the first benchmark to evaluate LLMs' ability to implement new features within existing repositories. More recently, the challenge of from-scratch repository generation has gained significant attention. Although CodeS (Zan et al., 2025b) and Commit0 (Zhao et al., 2024) automate repository collection from GitHub, these approaches struggle with scalability and are inherently susceptible to data leakage. While NL2Repo (Ding et al., 2025) and EvoCodeBench (Zhang et al., 2026) provide manually curated natural-language descriptions and test suites for Python repositories, and E2EDevBench (Liu et al., 2025b) utilizes GitHub-sourced repositories with "LLM-as-a-judge" rubrics, a fundamental bottleneck remains: the difficulty of constructing comprehensive test cases for repository-level generation without intensive human labor. To address this, we present a novel benchmark that evaluates an LLM agent's ability to generate a repository from scratch by re-implementing it based on the source APIs. Our framework represents the first benchmark that facilitates both automated test-case verification and large-scale production, ensuring both evaluation and scalability.

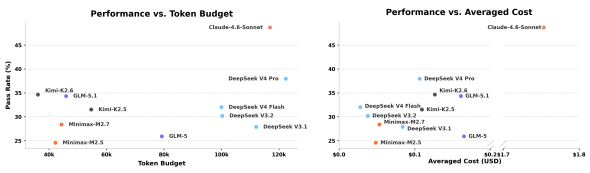

Figure 3: **Left:** pass rate vs. average token budget. **Right:** pass rate vs. average cost in USD. All models are evaluated with OpenHands-bash.

6.2 LLM Coding Agents

Following the reasoning-action loop (Yao et al., 2022), SWE-Agent (Yang et al., 2024a), and OpenHands (Wang et al., 2024) are the first LLM agents to conduct coding operations within a repository.

LocAgent (Chen et al., 2025), CoSIL (Jiang et al., 2025), and GraphLocator (Liu et al., 2025c) build a call graph for the repository and implement graph operations to assist repo-level reasoning. RepoSearcher (Ma et al., 2025), RepoNavigator (Zhang et al., 2025), and CodeScout (Sutawika et al., 2026) apply reinforcement learning to train the agent for bug fixing. The aforementioned agents are mainly designed for bug fixing, which is verifiable via unit tests. For code generation from scratch, as far as we know, there are only a limited number of agents that are specialized for the task. CodeS (Zan et al., 2025b) and RPG (Luo et al., 2025) perform repository generation by first designing the sketch of the repository, then filling in the code. The difficulty for rule-based verification is one of the most critical reasons that hinder the development of agents that generate repositories from scratch.

7 Conclusion

This paper presents RepoZero, the first scalable and verifiable benchmark for end-to-end repository generation. Unlike traditional code-completion tasks, RepoZero requires models to construct entire software projects from scratch. To ensure evaluation integrity and mitigate data contamination, we introduce a cross-language protocol across two primary subsets, *C2Rust* and *Py2JS*, encompassing 600 test files.

Our experimental analysis reveals that even state-of-the-art models, supported by advanced scaffolds, achieve only moderate success (approximately 40%), and self-verification is an important technique. These findings underscore that while the iterative "code-test" loop is a promising paradigm, a significant gap remains in agents' self-verification capabilities. Future progress in repository-level generation will necessitate a more robust integration of autonomous, high-quality test suite synthesis.

Limitations and Future Work We propose three primary directions to advance this field: (1) developing specialized training paradigms, such as reinforcement learning or distillation, to enhance global repository-level reasoning; (2) devising metrics and constraints to improve the architectural readability and structural integrity of generated codebases; and (3) expanding the benchmark to include large-scale, multilingual, and industry-level software development scenarios.

References

Anthropic. 2026. Introducing claude sonnet 4.6. https://www.anthropic.com/news/claude-sonnet-4-6. Accessed: 2026-04-23.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025. Locagent: Graph-guided llm agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8697–8727.

DeepSeek-AI. 2025. Deepseek-v3.1 release. https://api-docs.deepseek.com/news/news250821. Accessed: 2026-04-29.

DeepSeek-AI. 2026. Introducing deepseek v4. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro. Accessed: 2026-04-23.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, and 1 others. 2025. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*.

Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, Weiran Shi, Zaiyuan Wang, Daoguang Zan, Chenchen Zhang, Xiaoxu Zhang, Qizhi Chen, Xianfu Cheng, Bo Deng, Qingshui Gu, and 30 others. 2026. Nl2repo-bench: Towards long-horizon repository generation evaluation of coding agents. *Preprint*, arXiv:2512.12730.

Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, and 1 others. 2025. Nl2repo-bench: Towards long-horizon repository generation evaluation of coding agents. *arXiv preprint arXiv:2512.12730*.

Pengfei Gao, Zhao Tian, Xiangxin Meng, Xinchen Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, and 1 others. 2025. Trae agent: An llm-based agent for software engineering with test-time scaling. *arXiv preprint arXiv:2507.23370*.

Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I Wang. 2024. Cruxeval: A benchmark for code reasoning, understanding and execution. *arXiv preprint arXiv:2401.03065*.

Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. Cosil: Software issue localization via llm-driven code repository graph searching. *arXiv e-prints*, pages arXiv–2503.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? *Preprint*, arXiv:2310.06770.

Wei Li, Xin Zhang, Zhongxin Guo, Shaoguang Mao, Wen Luo, Guangyue Peng, Yangyu Huang, Houfeng Wang, and Scarlett Li. 2025. FEA-bench: A benchmark for evaluating repository-level code generation for feature implementation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17160–17176, Vienna, Austria. Association for Computational Linguistics.

Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, and 1 others. 2025a. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*.

Jingyao Liu, Chen Huang, Zhizhao Guan, Wenqiang Lei, and Yang Deng. 2025b. E2edev: Benchmarking large language models in end-to-end software development task. *arXiv preprint arXiv:2510.14509*.

Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. 2025c. Graphlocator: Graph-guided causal reasoning for issue localization. *arXiv preprint arXiv:2512.22469*.

Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, and 1 others. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664*.

Jane Luo, Xin Zhang, Steven Liu, Jie Wu, Jianfeng Liu, Yiming Huang, Yangyu Huang, Chengyu Yin, Ying Xin, Yuefeng Zhan, and 1 others. 2025. Rpg: A repository planning graph for unified and scalable codebase generation. *arXiv preprint arXiv:2509.16198*.

Zexiong Ma, Chao Peng, Qunhong Zeng, Pengfei Gao, Yanzhen Zou, and Bing Xie. 2025. Tool-integrated reinforcement learning for repo deep search. *arXiv preprint arXiv:2508.03012*.

Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, and 1 others. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*.

MinimaxAI. 2026. Introducing minimax. https://huggingface.co/MiniMaxAI/MiniMax-M2.7. Accessed: 2026-04-23.

MoonShot. 2026. Introducing kimi. https://huggingface.co/moonshotai/Kimi-K2.6. Accessed: 2026-04-23.

Qi Qian, Chengsong Huang, Jingwen Xu, Changze Lv, Muling Wu, Wenhao Liu, Xiaohua Wang, Zhenghua Wang, Zisu Huang, Muzhao Tian, and 1 others. 2026. Benchmarkˆ 2: Systematic evaluation of llm benchmarks. *arXiv preprint arXiv:2601.03986*.

Lintang Sutawika, Aditya Bharat Soni, Apurva Gandhi, Taha Yassine, Sanidhya Vijayvargiya, Yuchen Li, Xuhui Zhou, Yilin Zhang, Leander Melroy Maben, Graham Neubig, and 1 others. 2026. Codescout: An effective recipe for reinforcement learning of code search agents. *arXiv preprint arXiv:2603.17829*.

Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, and 1 others. 2026. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*.

Haifeng Wang, Hua Wu, Tian Wu, Yu Sun, Jing Liu, Dianhai Yu, Yanjun Ma, Jingzhou He, Zhongjun He, Dou Hong, and 1 others. 2026. Ernie 5.0 technical report. *arXiv preprint arXiv:2602.04705*.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, and 1 others. 2024. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*.

Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, and 1 others. 2026. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. 2024a. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, and 1 others. 2024b. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.

Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. 2025a. Multi-swe-bench: A multilingual benchmark for issue resolving. *Preprint*, arXiv:2504.02605.

Daoguang Zan, Ailun Yu, Wei Liu, Dong Chen, Bo Shen, Yafen Yao, Wei Li, Xiaolin Chen, Yongshun Gong, Bei Guan, Zhiguang Yang, Yongji Wang, Lizhen Cui, and Qianxiang Wang. 2025b. Codes: Natural language to code repository via multi-layer sketch. *ACM Trans. Softw. Eng. Methodol.* Just Accepted.

Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chengxing Xie, Cunxiang Wang, and 1 others. 2026. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*.

Wentao Zhang, Jianfeng Wang, Liheng Liang, Yilei Zhao, HaiBin Wen, and Zhe Zhao. 2026. Evocodebench: A human-performance benchmark for self-evolving llm-driven coding systems. *Preprint*, arXiv:2602.10171.

Zhaoxi Zhang, Yitong Duan, Yanzhi Zhang, Yiming Xu, Zhixiang Wang, Kun Liang, Yang Li, Jiahui Liang, Deguo Xia, Jizhou Huang, and 1 others. 2025. One tool is enough: Reinforcement learning for repository-level llm agents. *arXiv preprint arXiv:2512.20957*.

Wenting Zhao, Nan Jiang, Celine Lee, Justin T Chiu, Claire Cardie, Matthias Gallé, and Alexander M Rush. 2024. Commit0: Library generation from scratch. *arXiv preprint arXiv:2412.01769*.

A A Deeper Look into ACE

Table 4 reports the success rates achieved through the ACE workflow. A comparative analysis of the *Retry-1* and *Retry-2* iterations reveals that while the individual test case success rate exhibits only a marginal increase, the overall sample-level pass rate improves significantly (see Table 3). This divergence suggests that the ACE framework is particularly effective at resolving "corner case" failures within samples; by rectifying these pivotal edge cases, the framework disproportionately enhances the aggregate pass rate of entire repositories.

Building upon the empirical results of ACE, we propose two strategic directions for future research:

- **Comprehensive Test Suite Synthesis**: For agent-based software development, it is imperative to construct exhaustive test suites. This enables coding agents to leverage the ACE framework for rigorous self-verification, ensuring that generated codebases adhere to complex functional requirements.

- **High-Fidelity Predictive Oracles**: In scenarios where large-scale test cases are difficult to procure, developing advanced "oracle models" is essential. Such models must possess the capacity to precisely predict execution outputs given specific inputs and source code. Integrating these specialized testing models into the ACE workflow could substantially augment the reasoning and self-correction capabilities of coding agents.

B Prompts

We provide the prompt templates used in our agentic evaluation and ACE loop. Placeholders enclosed by angle brackets are instantiated for each task with the corresponding source file, target path, and repository-specific dependency constraints.

System Prompt

```
[system]
You are an AI expert with Linux terminal access. Use your
      reasoning ability before calling any tools.

[tool]
execute_shell(command): Execute a Linux shell command and
      return its output.
```

ACE Test Generation Prompt

```
[user]
Read the code and save test parameters to <
      TEST_CASES_JSON_PATH> as a JSON array:

You are a test engineer. Read the following Python code
      and generate 5 different sets of command-line
      arguments to test its logic.
Cover: normal input, boundary values, and possible error
      inputs.

Source code:
<PYTHON_SOURCE_CODE>

Output a JSON array of strings only, e.g.: ["--input 1",
      "--input 10 --verbose", ""]
No explanations, no output values -- only input argument
      strings.
```

ACE Refinement Prompt Suffix

```
[user]
<PY2JS_GENERATION_PROMPT>

[IMPORTANT] Do not rewrite from scratch -- fix the
      existing code.
Current path: <OUTPUT_MJS_PATH>
Library path: <OUTPUT_BASE>/packages/<
      RELATIVE_PATH_WITHOUT_PY>_pkg
Previous failure:
<FAILURE_MESSAGE>
```

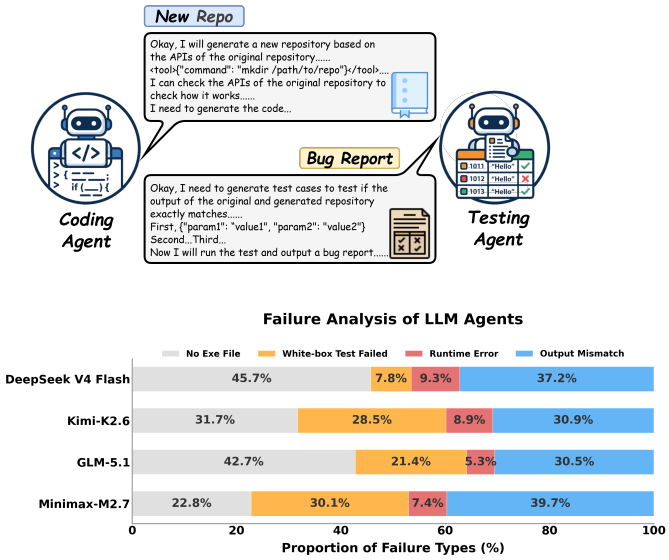

Figure 4: **Left:** an example illustrating the agentic code-test evolution framework. **Right:** analysis of failure conditions of RepoZero.

Table 4: Evaluation on **Py2JS** with DeepSeek V3.1 as the backbone model. We present the averaged success rate across all test cases across Easy, Medium, and Hard task difficulties, and the maximum retry times are set to 0 (coding only), 1 (coding-testing-refining), and 2 (coding-testing-refining-testing-refining).

Method	**Openhands-bash**				**Mini-SWE-Agent**			
	Easy	Medium	Hard	**Avg.**	Easy	Medium	Hard	**Avg.**
Retry-0	55.93	45.51	36.79	46.10	77.15	67.07	52.19	65.60
Retry-1	65.85	63.74	54.31	61.43	81.21	73.72	61.70	72.32
Retry-2	67.50	58.57	59.69	61.64	78.56	72.66	67.53	72.93